



42. Apriori Algorithm

"If it's frequent, its subsets are too"

Previous Section:

 [41. Gaussian Mixture Models](#)

Contents:

[1. Apriori Algorithm – The Overview](#)

[2. Apriori Algorithm – The Oversimplified Workflow](#)

[3. Apriori Algorithm with Python](#)

[Summary](#)

[References](#)

If you have never heard of the Apriori Algorithm, that's completely fine. I didn't either, until I went digging into Rule Learning a bit deeper. And somewhere along that path, between Rule-Based Classifiers and algorithms like RIPPER, this one quietly showed up.

At first, it doesn't look impressive. Just another algorithm. Another name.

But here's the thing, up until now, most of what we've seen in Machine Learning has been about letting the model *learn patterns on its own*. We give it data, it figures things out. No explicit rules. No direct instructions. Apriori flips that idea. Instead of hiding the rules inside a model, it **brings the rules to the surface**.

Think about it like this. Imagine you are analyzing shopping data. Thousands of transactions. People buying things every day. And inside all of that, there are patterns. Not obvious ones—but meaningful ones:

“People who buy bread... also buy butter.” ; “Customers who purchase a laptop... often buy a mouse.”

No one told the system these rules. They are *discovered*.

And that’s exactly where Apriori lives → **Rule Learning**. Not prediction. Not classification. But something more interpretable:

“IF this happens... THEN that is likely to happen.”

And suddenly, the model doesn’t feel like a black box anymore. It feels understandable.

So before we dive into the mechanics, keep this in mind: We are not trying to build a model that predicts. We are trying to uncover relationships that were always there, just hidden in the data.

1. Apriori Algorithm – The Overview

Rule Learning sounds simple, and honestly, it is. We are trying to learn rules. Not hidden patterns buried inside weights and matrices, but actual, readable logic:

“IF this happens... THEN that tends to happen.”

You’ve already seen this idea before with things like RIPPER. But now, we’re stepping into one of the most classic algorithms in this space: **Apriori**.

At its core, Apriori is not trying to predict a label or fit a curve. It’s doing something much more... observational. It looks at data—and asks: “What tends to appear together?”

Think about a real-world scenario. A restaurant, for example. Over time, it collects thousands of orders. And inside that data... there are patterns hiding in plain sight:

“Customers who order beef steak... often order french fries.”

Now here’s the key insight: No one programmed that rule. The algorithm *discovered* it. And once you know that? You can act on it:

- Bundle them as a combo → Increase sales;
- Place them together on the menu → Influence decisions;

- Or even design new dishes → Based on customer behavior.

This is where Apriori becomes powerful. Not because it is complex, but because it is **useful**.

So how does it actually work? Let's walk through the idea step by step.

We begin with a dataset. Imagine a long list of transactions—each one containing a set of items. These could be products, ingredients, anything.

- Now the algorithm starts very simply: It looks at **individual items**. Each item is treated as a group by itself → We call these **1-itemsets**. And then it counts. *"How often does each item appear?"*

But here's the important part. Not everything matters. Some items appear rarely. Some appear often. So we introduce a threshold: **Minimum Support**.

This is the rule that decides what is *important enough* to keep.

- If an item appears frequently enough (above the threshold) → it becomes a **frequent itemset**
- If not → it gets discarded

Already, we are filtering the noise.

Now comes the interesting part. Once we know which single items are frequent, we start combining them.

First, we form **pairs** (2-itemsets). Then we check again: *"Do these pairs appear frequently enough?"*

- If yes → keep them
- If not → discard them

Then we go further: 3-itemsets → 4-itemsets → and so on...

Step by step, growing the combinations. Until one day, nothing new survives. That's when we stop.

Now here's where Apriori becomes *clever*. It uses a simple—but powerful idea:

"If a small group is not frequent... any larger group containing it will also not be frequent."

Think about that for a second. If people *don't* buy bread and butter together... then there is no way they frequently buy *bread, butter, and milk* together. So

instead of checking everything blindly... the algorithm **cuts off entire branches early**. This is called the **Apriori Principle**.

And this one idea? → It saves a massive amount of computation.

Finally, once we have all the frequent itemsets. We turn them into actual rules:

IF A happens → THEN B happens

But not all rules are equally useful. So we evaluate them using:

- **Support** → How often does this rule appear?
- **Confidence** → How reliable is this rule?
- **Lift** → Is this relationship meaningful, or just coincidence?

And from there, we keep the strongest ones.

So if you zoom out, the whole idea becomes surprisingly clear: Apriori doesn't try to predict the future. It tries to *understand co-occurrence in the past*.

And from that, it gives you something incredibly valuable: Clear, interpretable rules, that you can actually use.

2. Apriori Algorithm – The Oversimplified Workflow

Enough theory for now. Let's actually see Apriori in action.

Imagine we are analyzing a fine-dining restaurant menu. Each dish is a "transaction", and each ingredient inside it is an "item".

Here's our dataset:

Dish	Ingredients
Carbonara	Pasta, Bacon, Egg, Parmesan
Chicken Tikka Masala	Chicken, Tomato, Yogurt, Garlic
Tacos	Tortilla, Beef, Tomato, Cheese
Sushi	Rice, Seaweed, Fish
Lasagna	Pasta, Beef, Tomato, Cheese
Pad Thai	Noodles, Egg, Tofu, Peanut
Falafel	Chickpea, Garlic, Herb

Dish	Ingredients
Ramen	Noodles, Egg, Pork
Paella	Rice, Seafood, Saffron
Beef Stroganoff	Beef, Mushroom, Cream

Our goal? Find which ingredients tend to appear together.

Step 1: Generate 1-Itemsets

We start simple. Just count how many times each ingredient appears.

Item	Count
Pasta	2
Bacon	1
Egg	3
Parmesan	1
Chicken	1
Tomato	3
Yogurt	1
Garlic	2
Tortilla	1
Beef	3
Cheese	2
Rice	2
Seaweed	1
Fish	1
Noodles	2
Tofu	1
Peanut	1
Chickpea	1
Herb	1
Pork	1
Seafood	1
Saffron	1

Item	Count
Mushroom	1
Cream	1

Now we apply a rule: **Minimum Support = 3** → So we only keep items that appear at least 3 times. That gives us: *Egg, Tomato, Beef*. These are our **frequent 1-itemsets**.



Here we use *support count* (raw frequency). In real systems, we usually use *support percentage*, because datasets can be huge.

Step 2: Generate 2-Itemsets

Now we combine the frequent items.

Possible pairs: (*Egg, Tomato*); (*Egg, Beef*); (*Tomato, Beef*)

But before blindly checking everything, we apply a helpful shortcut. We **filter the dataset** to only keep relevant ingredients:

Dish	Relevant Ingredients
Carbonara	Egg
Chicken Tikka Masala	Tomato
Tacos	Beef, Tomato
Lasagna	Beef, Tomato
Pad Thai	Egg
Ramen	Egg
Beef Stroganoff	Beef

Now we count how often each pair appears:

2-Itemset	Support Count
{Egg, Tomato}	0
{Egg, Beef}	0
{Tomato, Beef}	2

And here's the problem: **Minimum Support = 3** → None of these reach it
So no frequent 2-itemsets, no larger itemsets → The algorithm **stops here**

Step 3: Generate Association Rules

Even though we didn't get larger itemsets, we can still form rules from what we have. This is where Apriori becomes practical. We define:

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- $X \rightarrow Y$ means that if a transaction contains X , it is likely to also contain Y .
- $\text{Support}(X \cup Y)$ is the number of transactions containing both X and Y .
- $\text{Support}(X)$ is the number of transactions containing X .

Now let's try an example: **Beef** → **Tomato**

We know:

- $\text{Support}(\text{Beef}) = 3$
- $\text{Support}(\text{Beef} \cup \text{Tomato}) = 2$

So: $\text{Confidence} = 2/3 = 0.67$

Meaning: **67% of dishes that contain Beef also contain Tomato**

At the end of all this... Apriori gives us something very different from most ML models: Not predictions. Not probabilities over classes.

But **clear relationships**:

- Support → "How often does this happen?"
- Confidence → "How reliable is this rule?"
- Lift → "Is this actually meaningful... or just coincidence?"

And in real-world applications, we go one step further: We set a **minimum confidence threshold**. So only strong, reliable rules survive.

If you step back for a moment... the whole workflow is surprisingly intuitive:

Start small → Filter → Combine → Prune → Extract rules

And just like that, hidden patterns in data become **decisions you can actually use**.

3. Apriori Algorithm with Python

Understanding Apriori is one thing. **Actually running it?** → That's where it clicks. We'll use a dataset of dishes and ingredients:

dishes_apriori_dataset.csv

- Each dish = one transaction
- Each ingredient = one item
- First, we read the dataset and group ingredients by each dish.

```
import pandas as pd

df = pd.read_csv('dishes_apriori_dataset.csv')

# Group items purchased together by the same dishes
basket = df.groupby(['Dish'])['Ingredient'].apply(list).reset_index()
transactions = basket['Ingredient'].tolist()
```

At this point, the data looks like a 2D array conceptually. But Apriori can't work with this format yet.

- Apriori needs a **binary matrix**: *Did an ingredient appear?* → *True / False*. So we use a **Transaction Encoder**.

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```

from mlxtend.preprocessing import TransactionEncoder

# Apriori needs data in True/False format
encoder = TransactionEncoder()
encoded_array = encoder.fit(transactions).transform(transac
tions)
df_encoded = pd.DataFrame(encoded_array, columns=encoder.co
lums_)

```

Now the data becomes something like a True/False table whose columns represent the unique ingredients appearing in the original dataset. This is exactly what Apriori needs.

Now we let the algorithm do its job → **Generate Frequent Itemsets**

```

frequent_itemsets = apriori(df_encoded, min_support=0.2, us
e_colnames=True)
print("Total Frequent Item-sets:", frequent_itemsets.shape
[0])

```

Total Frequent Item-sets: 13

`min_support = 0.2` means: "Keep itemsets that appear in at least 20% of dishes"

We extract rules from those itemsets → **Generate Association Rules**

```

rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.1)

rules = rules[rules['antecedents'].apply(lambda x: len(x) >= 1) &
              rules['consequents'].apply(lambda x: len(x) >= 1)]

print("Association Rules:", rules.shape[0])
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])

```

You can run this code on any Python Environment for the clear result

Understanding the Output

From the results:

- We obtained **13 frequent itemsets**
 - 1-itemsets → common ingredients
 - 2-itemsets → ingredients that appear together
 - 3-itemsets → stronger combinations (e.g., Tacos & Lasagna pattern)
- We obtained **14 association rules**

Now here's where it gets interesting. Some rules are **perfect: Confidence = 1.0**

Meaning: Whenever X appears, Y *always* appears

Examples:

- Cheese → Beef
- Noodles → Egg
- Beef, Tomato → Cheese

These are not guesses. They are **guarantees within this dataset**.

Understanding Lift - Don't ignore this one.

Lift tells us: "Is this relationship actually meaningful... or just coincidence?"

- **Lift = 1** → No relationship
- **Lift > 1** → Positive relationship
- **Lift >> 1** → Strong relationship

So if: **Lift = 5.0** → That means these items appear together **5× more often than random chance**

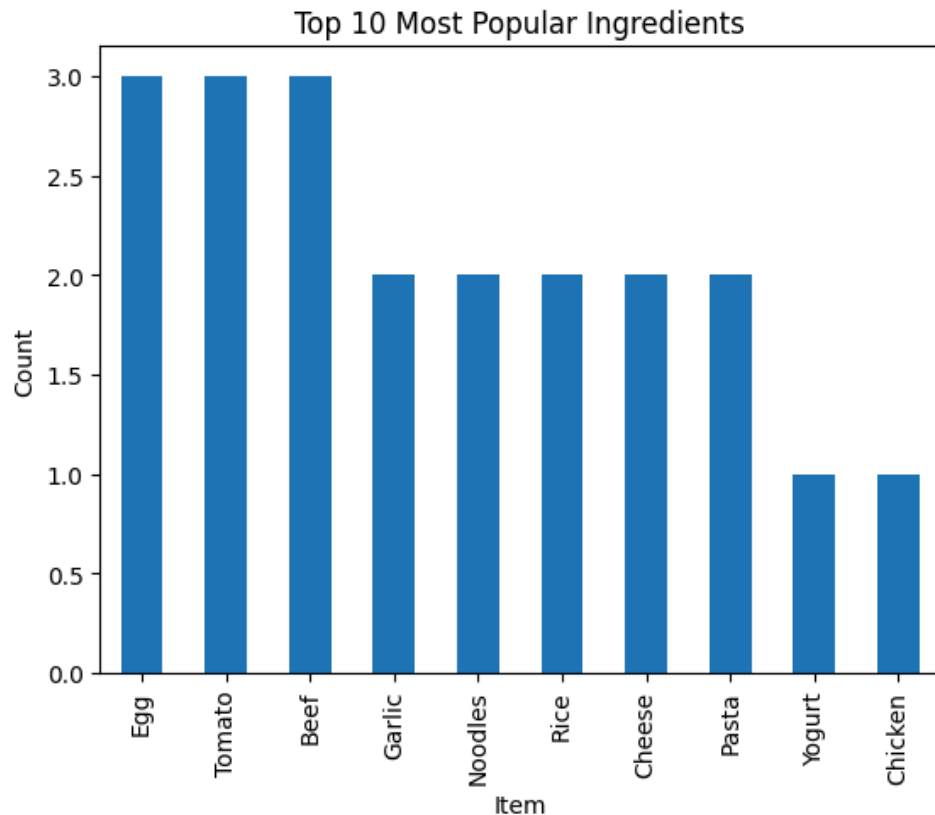
That's not random anymore. That's a pattern.

- Let's quickly visualize the most popular ingredients:

```
import matplotlib.pyplot as plt

top_items = df['Ingredient'].value_counts().head(10)

top_items.plot(kind='bar', title='Top 10 Most Popular Ingredients')
plt.xlabel("Item")
plt.ylabel("Count")
plt.show()
```



To be honest, the result does not surprise me... at all!!

This gives a simple but powerful insight: ***What ingredients dominate the dataset?***

Summary

Apriori is one of those algorithms that doesn't look impressive at first. No trees. No neural networks. No optimization tricks. No fancy math. And yet, it does something many models don't:

It gives you **clear, human-readable knowledge**.

Instead of saying: "Prediction = 0.87" → It tells you: *"If this happens... that tends to happen."*

That's powerful. Because in real-world systems—especially business, marketing, and recommendation—you don't just want predictions. You want **understanding**. Apriori gives you that. It finds patterns you didn't explicitly look for. It reveals relationships hidden in plain sight. And most importantly... It turns data into **decisions you can actually act on**.

Some says it is often overlooked by learners? Yes. But once you see what it's doing... you realize it was never simple. It was just... **honest about the patterns all along.**

References

<https://www.geeksforgeeks.org/machine-learning/apriori-algorithm/>

<https://www.geeksforgeeks.org/machine-learning/implementing-apriori-algorithm-in-python/>

<https://www.datacamp.com/tutorial/apriori-algorithm>

<https://www.ibm.com/think/topics/apriori-algorithm>

<https://www.mygreatlearning.com/blog/apriori-algorithm-explained/>

<https://www.youtube.com/watch?v=guVvtZ7ZClw>

Next Section:

 43. Semi-Supervised Learning